

Searching



- Searching refers to the operation for finding something
- We have an array DATA, specific ITEM of information
- Searching will give us the LOC of ITEM
- Successful
 - ITEM appears in DATA
- Unsuccessful
 - ITEM does not appears in DATA

35=?-----Successful

100=?-----Unsuccessful

10
25
35
68
78
15

Searching Algorithm



- Depends on the way the information in DATA is organized.
- Linear Search: when the data is unsorted or sorted
- Binary Search: when the data is **sorted only!!!**

10
25
35
68
78
15

Linear Search

10
15
25
35
68
75

Binary Search

Linear Search



- We have an array DATA with N elements.
- We have to find the LOC of ITEM(Say ITEM=68)

Working Process:

10

Check element one by one.

25

DATA[1]=ITEM ???

35

YES then OK.

68

otherwise DATA[2]=ITEM ???

78

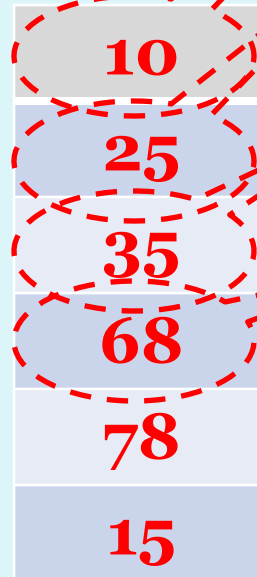
15

.....So on.

Linear Search



1. $K:=1$ and $LOC=0$.
2. Repeat Steps 3 and 4 while $K \leq N$
3. If $ITEM=DATA[K]$, then $LOC:=K$
4. Set $K:=K+1$
5. If $LOC=0$ then
-Write :ITEM is not Found
6. Else
-Write : ITEM Found at LOC
- 7.Exit.



1st: $K=1$ $DATA[1]=10$

$68 \neq 10$ so $K=2$

2nd: $K=2$ $DATA[2]=25$

$68 \neq 25$ so $K=3$

3rd: $K=3$ $DATA[3]=35$

$68 \neq 35$ so $K=4$

4th: $K=4$ $DATA[4]=68$

$68 == 68$

So $LOC=K=4$

Write: ITEM found at

Location 4

Binary Search



- We have an array DATA with n elements.
- Data or information must be sorted.
- Binary search is efficient than linear search.
- Think of a telephone directory
----you need to find **Nazim**
- Do you need to search a name from A to Z
- **Obviously not!!!!**

10
25
35
68
78
15

Binary Search



Conditions:

- Data or information must be sorted.
- Data or information must be into an array

Applicable ???? NO

10
105
25
68
78
85

Binary Search



Conditions:

- Data or information must be sorted.
- Data or information must be into an array

Applicable ????

Yes

10
15
25
68
78
85

Binary Search



- We have an array DATA with N elements.
- We have to find the LOC of ITEM(Say ITEM=60)

10	20	30	40	50	60	70	80	90	100
----	----	----	----	----	----	----	----	----	-----

BEG= 1st Position=1

END= Last Position=10

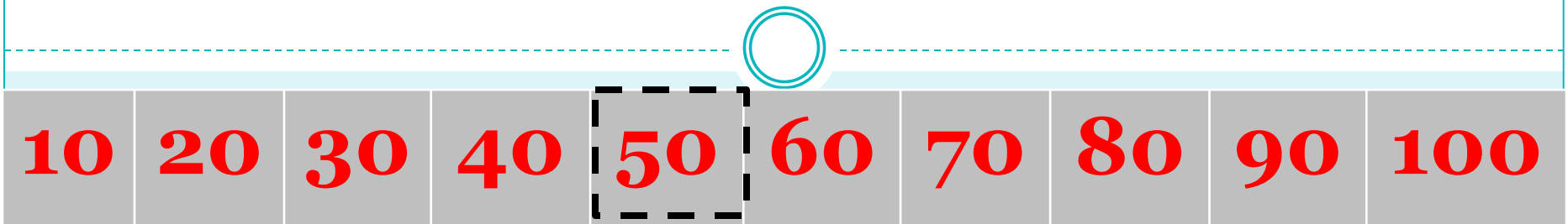
Binary Search



10 20 30 40 50 60 70 80 90 100

- We calculate the MID position.
$$\text{MID} = \text{INT}((\text{BEG} + \text{END}) / 2)$$
- Pick up the DATA[MID] and Compare with ITEM.
- If $\text{ITEM} > \text{DATA}[\text{MID}]$ then we search the right half
Reset **BEG:=MID+1**
- If $\text{ITEM} < \text{DATA}[\text{MID}]$ then we search the left half
Reset **END:=MID-1**
- If $\text{ITEM} = \text{DATA}[\text{MID}]$ then search Complete.
- Otherwise Not found.

Binary Search



10	20	30	40	50	60	70	80	90	100
----	----	----	----	----	----	----	----	----	-----

1. Set $LOC=0$ $MID:=INT((BEG+END)/2)$
2. Repeat Steps 3 and 4 while $BEG \leq END$
and $DATA[MID] \neq ITEM$
3. If $ITEM > DATA[MID]$
Set $BEG := MID + 1$
Else: Set $END := MID - 1$
4. $MID := INT((BEG + END) / 2)$
5. IF $DATA[MID] = ITEM$
Set $LOC := MID$
Else $LOC := NULL$;
- 6 .Exit.

$BEG = 1, END = 10, MID = 5$

1st: $DATA[MID] = DATA[5] = 50$

$50 \neq 60$

and $60 > 50$

So

$BEG = MID + 1 = 5 + 1 = 6$

$END = 10$

$MID := 8$

Binary Search



10	20	30	40	50	60	70	80	90	100
----	----	----	----	----	----	----	----	----	-----

1. Set $LOC=0$ $MID:=INT((BEG+END)/2)$
2. Repeat Steps 3 and 4 while $BEG \leq END$
and $DATA[MID] \neq ITEM$
3. If $ITEM > DATA[MID]$
Set $BEG := MID + 1$
Else: Set $END := MID - 1$
4. $MID := INT((BEG + END) / 2)$
5. IF $DATA[MID] = ITEM$
Set $LOC := MID$
Else $LOC := NULL$;
- 6 .Exit.

$BEG=6, END=10, MID=8$
2st: $DATA[MID] = DATA[8] = 80$

$80 \neq 60$
and $60 < 80$

So
 $END = MID - 1 = 8 - 1 = 7$
 $BEG = 6$
 $MID := 6$

Binary Search

10	20	30	40	50	60	70	80	90	100
----	----	----	----	----	----	----	----	----	-----

1. Set $LOC=0$ $MID:=INT((BEG+END)/2)$
2. Repeat Steps 3 and 4 while $BEG \leq END$
and $DATA[MID] \neq ITEM$
3. If $ITEM > DATA[MID]$
Set $BEG := MID + 1$
Else: Set $END := MID - 1$
4. $MID := INT((BEG + END) / 2)$
5. IF $DATA[MID] = ITEM$
Set $LOC := MID$
Else $LOC := NULL$;
- 6 .Exit.

$BEG=6, END=7, MID=6$
3rd: $DATA[MID] = DATA[6] = 60$
 $60 = 60$
So
 $LOC = 6$

Insertion Sort

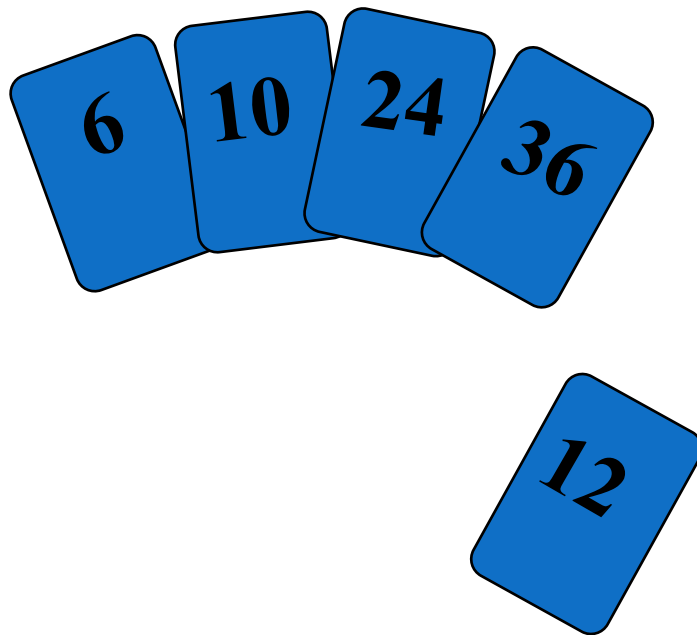


- Insertion sort is a simple sorting algorithm that is appropriate for small inputs.
 - Most common sorting technique used by card players.
- The list is divided into two parts: **sorted and unsorted.**
- In each pass, the first element of the unsorted part is picked up, transferred to the sorted sublist, and inserted at the appropriate place.
- A list of n elements will take at most $n-1$ passes to sort the data.

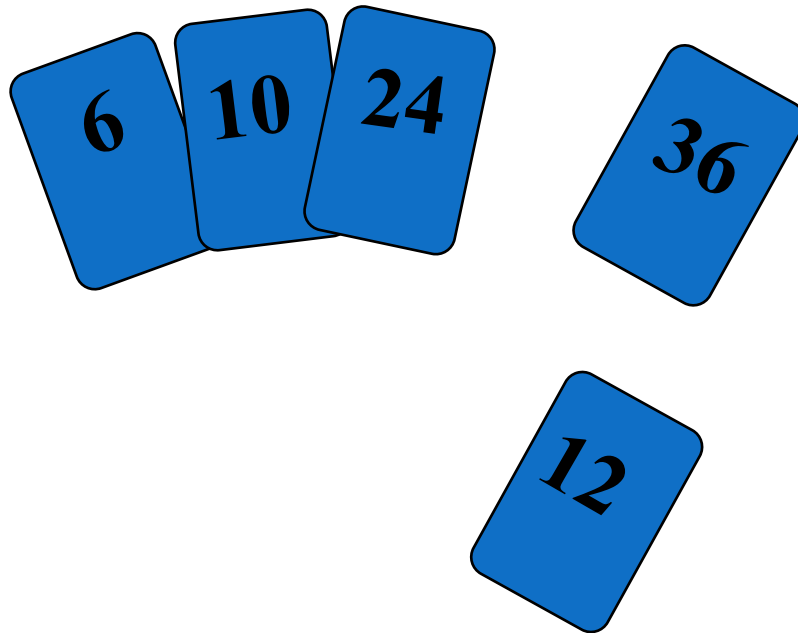
Insertion Sort



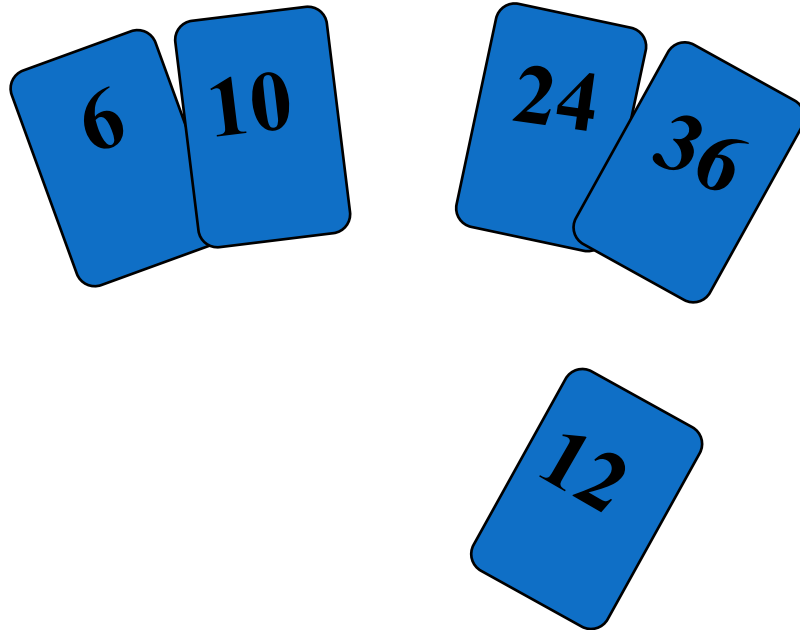
To insert 12, we need to make room for it by moving first 36 and then 24.



Insertion Sort



Insertion Sort



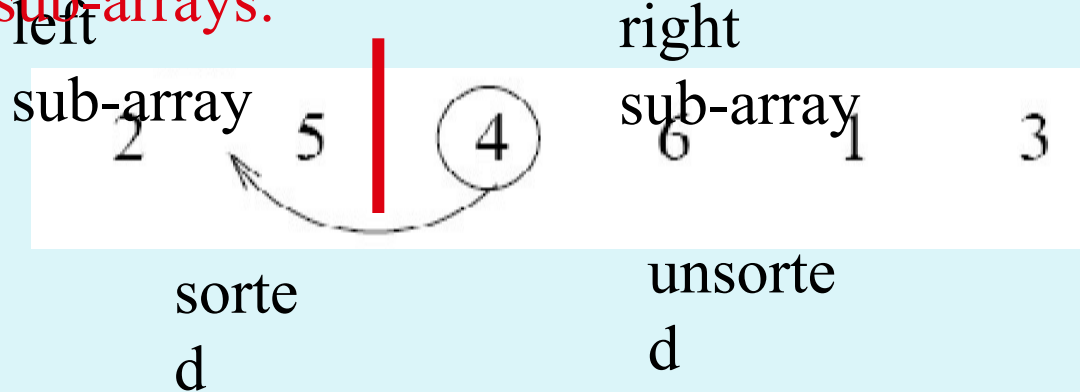
Insertion Sort



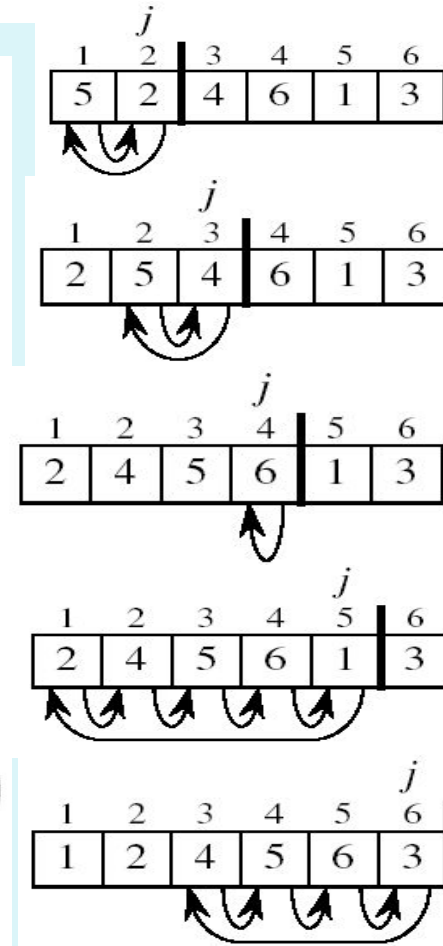
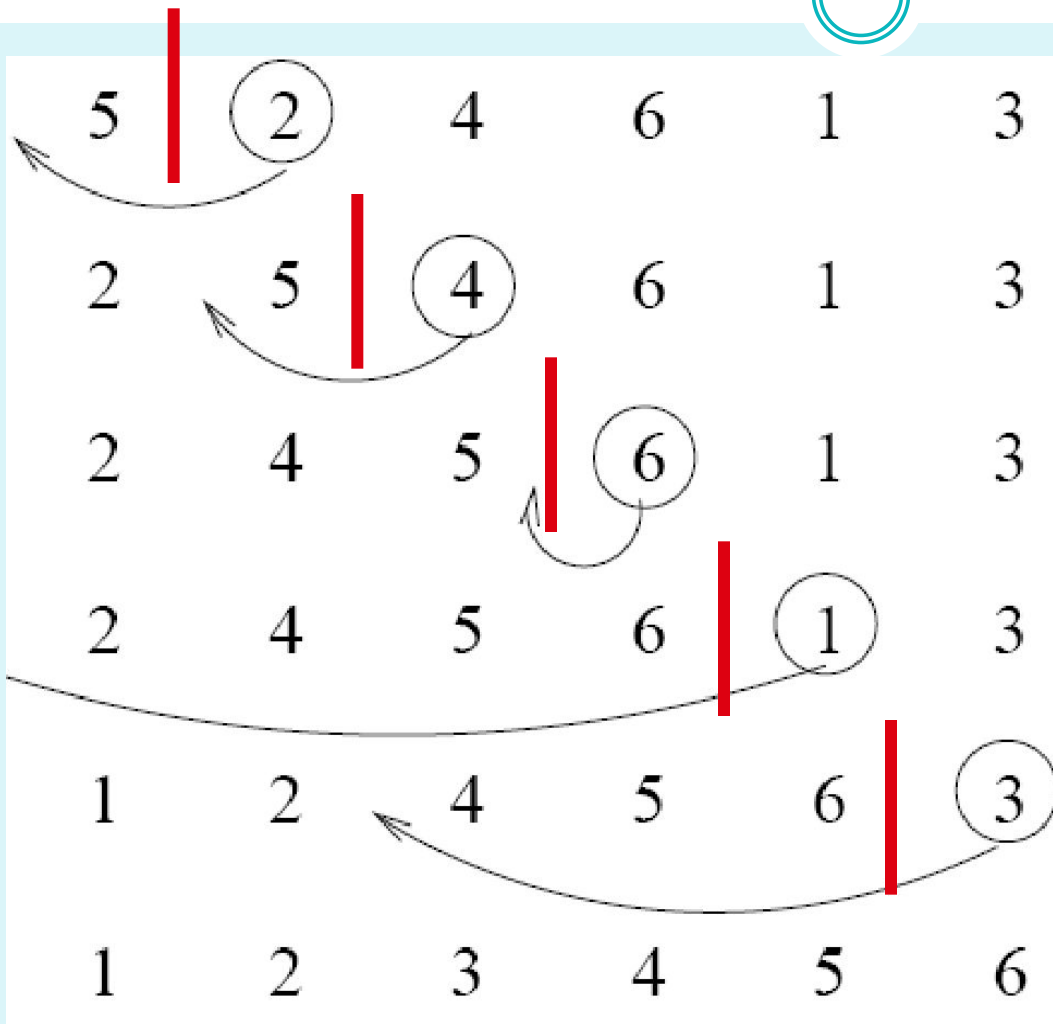
input

5 2 4 6 1 3

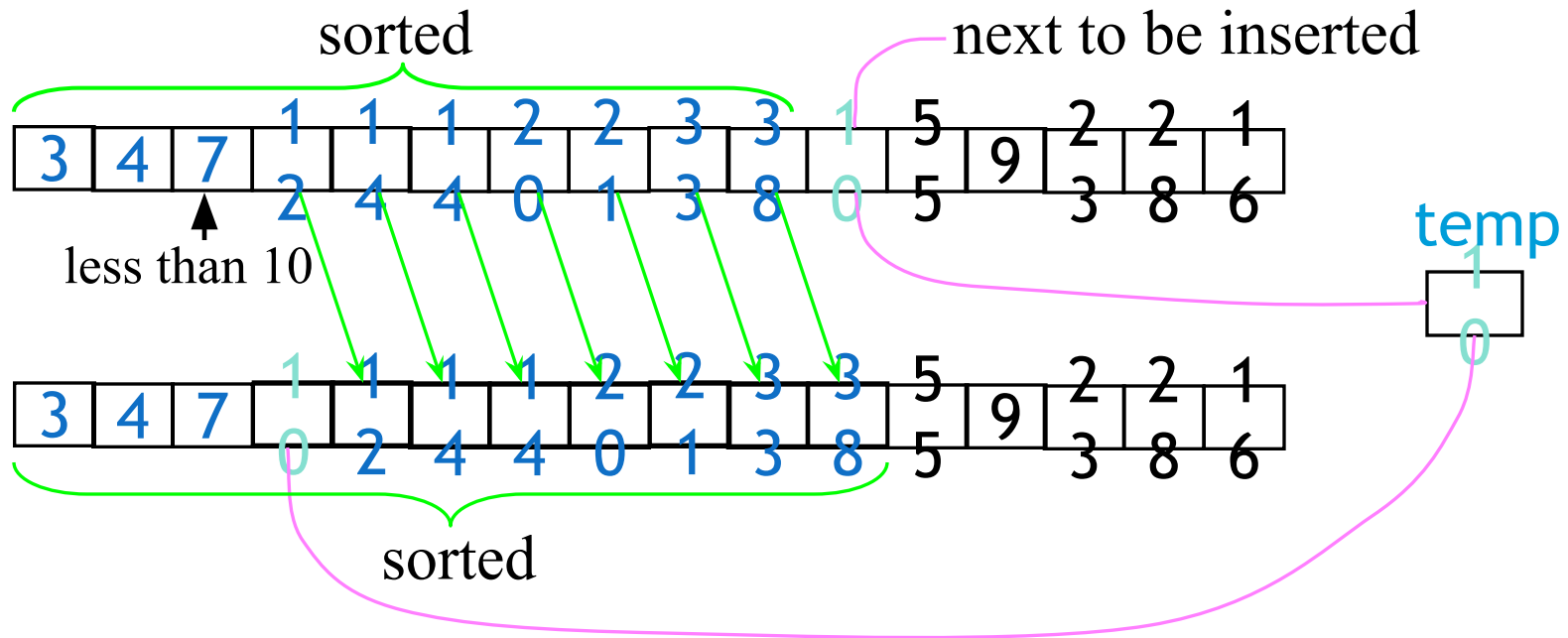
at each iteration, the array is divided in two sub-arrays:



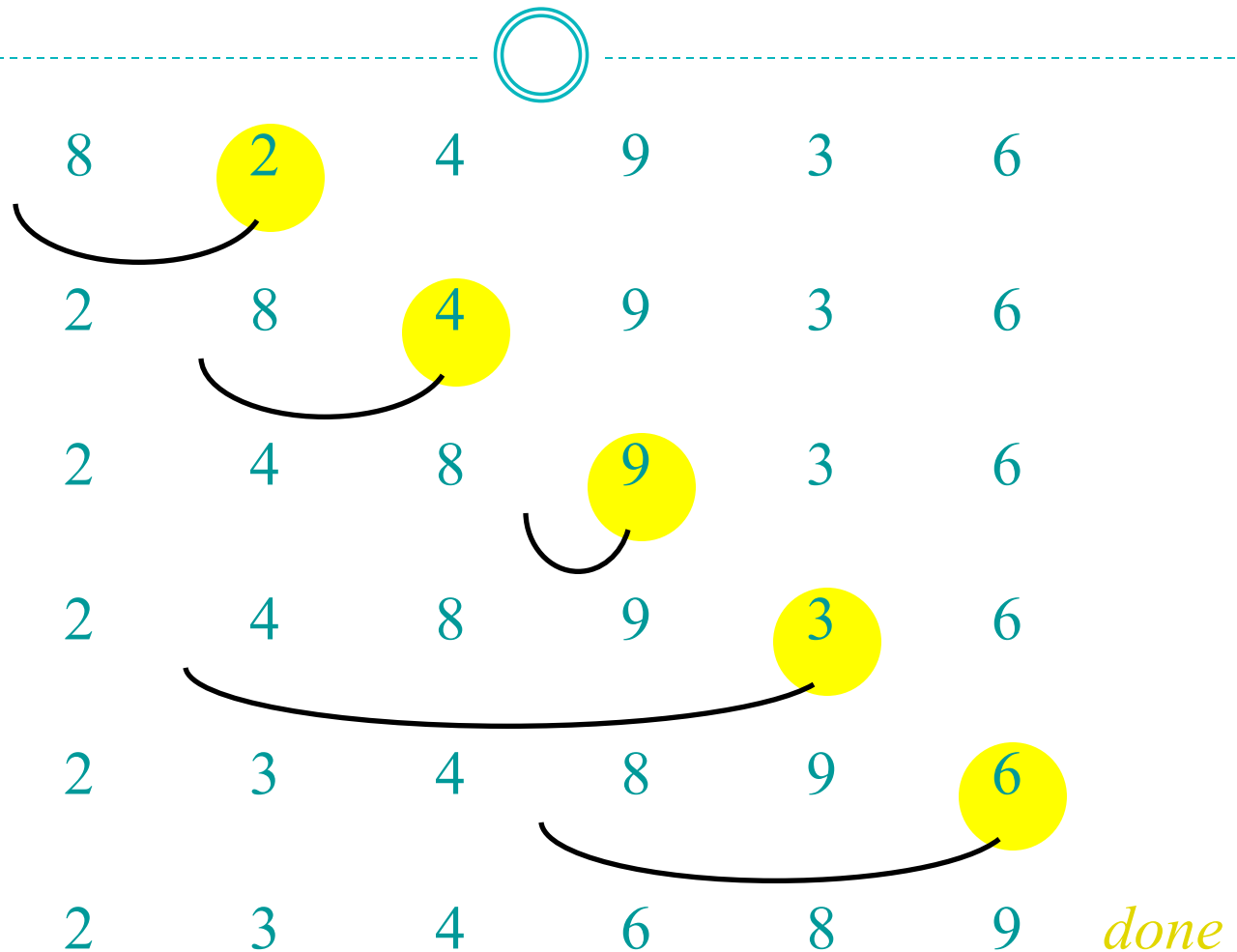
Insertion Sort



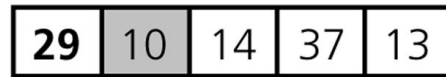
One step of insertion sort



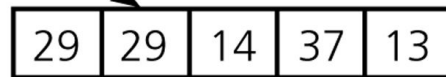
Example of insertion sort



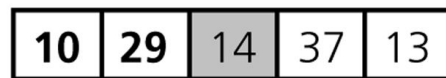
Initial array:



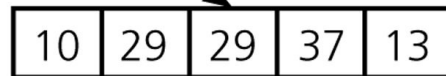
Copy 10



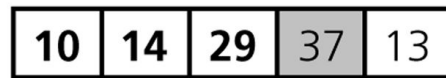
Shift 29



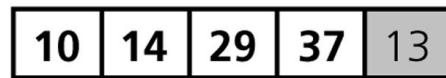
Insert 10; copy 14



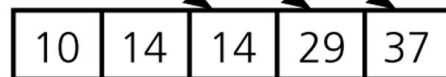
Shift 29



Insert 14; copy 37, insert 37 on top of itself

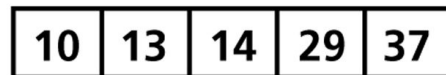


Copy 13



Shift 37, 29, 14

Sorted array:



Insert 13

An insertion sort of an array of five integers

Algorithm: Insertion_Sort(List , N)



Here List is the list of items and N is the total number of items.

1. Repeat steps 2 to 7 for $I = 2 \dots N$
2. Set $\text{Key} := \text{List}[I]$
3. Set $J := I - 1$
4. Repeat steps 5 and 6 while $J \geq 1$ and $\text{List}[J] > \text{Key}$
5. $\text{List}[J+1] := \text{List}[J]$
6. Decrement J
7. Set $\text{List}[J+1] := \text{Key}$;
8. End.

An Example: Insertion Sort



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

30	10	40	20
1	2	3	4

$i = \emptyset \quad j = \emptyset \quad \text{key} = \emptyset$
 $A[j] = \emptyset \quad A[j+1] = \emptyset$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

30	10	40	20
1	2	3	4

$i = 2$ $j = 1$ $\text{key} = 10$
 $A[j] = 30$ $A[j+1] = 10$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$ $j = 1$ $\text{key} = 10$
 $A[j] = 30$ $A[j+1] = 30$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2$ $j = 1$ $\text{key} = 10$
 $A[j] = 30$ $A[j+1] = 30$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2 \quad j = 0 \quad \text{key} = 10$
 $A[j] = \emptyset \quad A[j+1] = 30$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

30	30	40	20
1	2	3	4

$i = 2 \quad j = 0 \quad \text{key} = 10$
 $A[j] = \emptyset \quad A[j+1] = 30$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 2 \quad j = 0 \quad \text{key} = 10$
 $A[j] = \emptyset \quad A[j+1] = 10$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3 \quad j = 0 \quad \text{key} = 10$
 $A[j] = \emptyset \quad A[j+1] = 10$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$ $j = 0$ $\text{key} = 40$
 $A[j] = \emptyset$ $A[j+1] = 10$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$ $j = 0$ $\text{key} = 40$
 $A[j] = \emptyset$ $A[j+1] = 10$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$ $j = 2$ $\text{key} = 40$
 $A[j] = 30$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$ $j = 2$ $\text{key} = 40$
 $A[j] = 30$ $A[j+1] = 40$

```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 3$ $j = 2$ $\text{key} = 40$
 $A[j] = 30$ $A[j+1] = 40$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 40$
 $A[j] = 30$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 20$
 $A[j] = 30$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 20$
 $A[j] = 30$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$ $j = 3$ $\text{key} = 20$
 $A[j] = 40$ $A[j+1] = 20$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	20
1	2	3	4

$i = 4$ $j = 3$ $\text{key} = 20$
 $A[j] = 40$ $A[j+1] = 20$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$ $j = 3$ $\text{key} = 20$
 $A[j] = 40$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$ $j = 3$ $\text{key} = 20$
 $A[j] = 40$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$ $j = 3$ $\text{key} = 20$
 $A[j] = 40$ $A[j+1] = 40$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 20$
 $A[j] = 30$ $A[j+1] = 40$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	40	40
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 20$
 $A[j] = 30$ $A[j+1] = 40$



```
InsertionSort(A, n) {  
    for i = 2 to n {  
        key = A[i]  
        j = i - 1;  
        while (j > 0) and (A[j] > key) {  
            A[j+1] = A[j]  
            j = j - 1  
        }  
        A[j+1] = key  
    }  
}
```

An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 20$
 $A[j] = 30$ $A[j+1] = 30$



```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$ $j = 2$ $\text{key} = 20$
 $A[j] = 30$ $A[j+1] = 30$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$ $j = 1$ $\text{key} = 20$
 $A[j] = 10$ $A[j+1] = 30$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	30	30	40
1	2	3	4

$i = 4$ $j = 1$ $\text{key} = 20$
 $A[j] = 10$ $A[j+1] = 30$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	20	30	40
1	2	3	4

$i = 4$ $j = 1$ $\text{key} = 20$
 $A[j] = 10$ $A[j+1] = 20$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```



An Example: Insertion Sort

10	20	30	40
1	2	3	4

$i = 4$ $j = 1$ $\text{key} = 20$
 $A[j] = 10$ $A[j+1] = 20$

```
InsertionSort(A, n) {  
  for i = 2 to n {  
    key = A[i]  
    j = i - 1;  
    while (j > 0) and (A[j] > key) {  
      A[j+1] = A[j]  
      j = j - 1  
    }  
    A[j+1] = key  
  }  
}
```

Done!



Any Questions

????????



Thanks